

Preempção Síncrona vs Assíncrona em Sistemas Embarcados Críticos: O Caso do CDC-P

Marcos Roberto Braga

Instituto Informal de Educação, Ciência e Tecnologia

Departamento de Engenharia Eletrônica e Sistemas Embarcados

E-mail: noelmrb_tec@yahoo.com

Resumo—Os sistemas operacionais de tempo real (RTOS) comerciais adotam majoritariamente a preempção assíncrona como mecanismo de escalonamento. Esta abordagem, embora tolerante a erros de dimensionamento temporal, introduz sobrecarga significativa de memória, não-determinismo, e vulnerabilidades a condições de corrida, deadlocks e inversão de prioridade. O presente artigo demonstra, através de análise teórica e resultados experimentais, que a preempção síncrona — implementada no RTOS CDC-P (Concurrency Deterministic Control with Preemption) — é superior em consumo de recursos, previsibilidade temporal e imunidade a falhas de concorrência. Utilizando um microcontrolador PIC16F628A de 8 bits a 4 MHz, o CDC-P executa 10 tarefas com apenas 1360 palavras de ROM, 80 bytes de RAM e 4 níveis de pilha, entregando resposta imediata a eventos urgentes na mesma iteração do despachador (latência zero ticks). Argumenta-se que a predominância da preempção assíncrona na indústria não decorre de superioridade técnica, mas de um modelo de negócio que monetiza a complexidade e a dependência de ecossistemas proprietários.

I. INTRODUÇÃO

O desenvolvimento de software para sistemas embarcados críticos — dispositivos médicos, controle automotivo, aviônica e automação industrial — impõe requisitos rigorosos de confiabilidade, determinismo temporal e eficiência de recursos. Neste contexto, a escolha do sistema operacional de tempo real (RTOS) e de sua política de escalonamento não é apenas uma decisão técnica, mas uma questão de segurança.

A. O Estado da Arte

RTOS comerciais como FreeRTOS, ThreadX, VxWorks e QNX baseiam-se na preempção assíncrona: um temporizador de hardware interrompe a tarefa em execução em pontos arbitrários, transfere o controle ao escalonador, e este seleciona a próxima tarefa baseado em prioridade. Para preservar o estado da tarefa interrompida, todo o contexto (registradores, stack pointer, flags de status) precisa ser salvo em memória.

B. O Problema

Esta abordagem, embora amplamente adotada, gera consequências bem documentadas na literatura [1], [3], [4]:

- **Sobrecarga de memória:** cada tarefa requer sua própria pilha, tipicamente de 512 bytes a 4 KB;
- **Não-determinismo temporal:** a latência de resposta a eventos depende do ponto de interrupção e do estado do escalonador;

- **Vulnerabilidades:** condições de corrida, deadlocks e inversão de prioridade são inerentes ao modelo;
- **Complexidade paliativa:** mutexes, semáforos, priority inheritance e outras construções são adicionadas para mitigar problemas que o próprio modelo cria.

C. Objetivo

Este artigo apresenta o CDC-P (Concurrency Deterministic Control with Preemption), um RTOS de arquitetura cooperativa com preempção síncrona que elimina, por construção, as vulnerabilidades acima. Demonstra-se que a preempção síncrona não apenas é viável, mas tecnicamente superior em sistemas com restrições críticas de recursos e segurança.

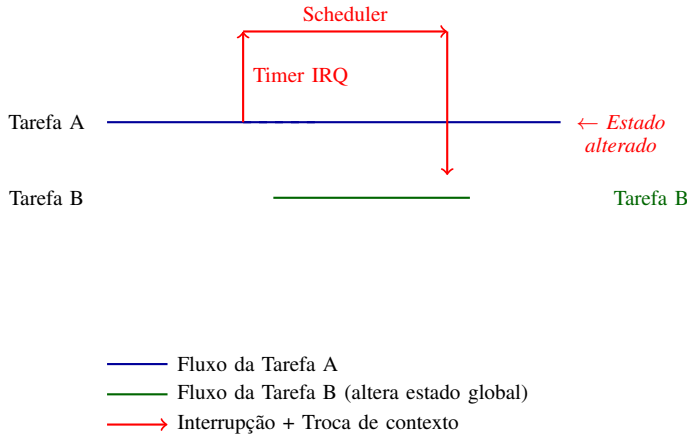
II. FUNDAMENTAÇÃO TEÓRICA

A. Dijkstra e o GOTO Prejudicial

Em 1968, Edsger W. Dijkstra publicou “Go To Statement Considered Harmful” [1], argumentando que o desvio incondicional (`goto`) corrompe a capacidade do programador de raciocinar sobre o estado do programa. Sem um fluxo de controle estruturado, torna-se impossível provar propriedades sobre o sistema.

B. GOTO Temporal: A Preempção como Ruptura de Fluxo

A preempção assíncrona constitui um **GOTO temporal**: o fluxo de controle é arrancado de seu ponto atual por um evento de timer, transportado ao escalonador, e posteriormente devolvido — mas em um ambiente que pode ter sido alterado por outras tarefas durante a ausência. O programador perde a capacidade de raciocinar sobre o estado do sistema entre duas instruções consecutivas de sua tarefa.



C. Sistemas Síncronos vs Assíncronos

Sistemas síncronos operam em pontos de sincronização bem definidos (bordas de clock, fim de iteração do despachador). Sistemas assíncronos permitem mudanças de estado a qualquer instante. A teoria de controle e a engenharia de confiabilidade demonstram que sistemas síncronos são intrinsecamente mais fáceis de analisar, verificar e certificar [6].

D. O Preço do Não-Determinismo

Em sistemas críticos, onde vidas humanas dependem do comportamento correto do software, o não-determinismo não é uma inconveniência — é uma ameaça. Cada condição de corrida é um bug que pode ou não se manifestar, dependendo de temporizações que variam com temperatura, tensão de alimentação e tolerâncias de fabricação.

III. PREEMPÇÃO ASSÍNCRONA NOS RTOS COMERCIAIS

A. Mecanismo

O escalonador preemptivo opera da seguinte forma:

- 1) Um temporizador de hardware (SysTick em ARM, timer0 em PIC) gera interrupções periódicas;
- 2) A ISR do temporizador invoca o escalonador;
- 3) O escalonador salva o contexto da tarefa atual (registros, stack pointer, program counter);
- 4) O escalonador seleciona a próxima tarefa baseado em prioridade;
- 5) O contexto da nova tarefa é restaurado;
- 6) O controle é transferido.

B. Consequências

C. Overhead Típico

Estimativas conservadoras para FreeRTOS em ARM Cortex-M [7]:

- ROM do kernel: 6–10 KB
- RAM por tarefa (pilha + TCB): 256–2048 bytes
- Latência de troca de contexto: 3–15 μs (dependendo da arquitetura)
- Jitter: variável, dependente do estado do sistema

Tabela I
PROBLEMAS GERADOS PELA PREEMPÇÃO ASSÍNCRONA

Problema	Descrição
Condições de corrida	Duas tarefas acessam a mesma variável; o resultado depende da ordem de interrupção
Deadlocks	Tarefa A aguarda recurso de B, B aguarda recurso de A
Inversão de prioridade	Tarefa de alta prioridade aguarda recurso segurado por tarefa de baixa prioridade
Stack overflow	Pilha insuficiente para o pior caso de aninhamento de interrupções
Jitter	Variação na latência de resposta a eventos

D. O Modelo de Negócio da Complexidade

A complexidade introduzida pela preempção assíncrona não é um acidente — é um mercado. Ferramentas de debug avançado (trace de eventos, analisadores lógicos), consultorias de certificação, treinamentos oficiais e licenças de RTOS comerciais constituem um ecossistema que fatura bilhões anualmente. A dependência de APIs proprietárias dificulta a migração entre fabricantes, criando aprisionamento tecnológico (*vendor lock-in*).

IV. O CDC-P E A PREEMPÇÃO SÍNCRONA

A. Arquitetura do Despachador

O CDC-P utiliza um despachador cooperativo com sequência fixa e imutável. A cada iteração (um tick, tipicamente 8 ms), o sistema executa, nesta ordem:

- 1) **Bloco URG-S:** tarefas com flag de urgência situacional (resposta imediata, mesma iteração);
- 2) **Bloco CDC-P:** tarefa com preempção cooperativa ativa (período reduzido a 1 tick);
- 3) **Camada de Supervisão:** Task10 (Xerife, diagnóstico de pane) e Task9 (Síndico, recuperação);
- 4) **Camada de Aplicação:** Task1 a Task8, em ordem fixa de prioridade espacial.

Cada tarefa executa completamente e **retorna** ao despachador. Não há interrupção de tarefa por timer. Não há salvamento de contexto. A pilha do sistema permanece em 4 níveis durante toda a operação.

B. Mecanismo URG-S (Urgência Situacional)

O URG-S proporciona resposta a eventos na **mesma iteração** do despachador, com latência zero ticks. Uma flag binária associada a cada tarefa é setada pela ISR do periférico. No início da próxima iteração, antes das tarefas normais, o despachador verifica as flags e executa imediatamente as tarefas sinalizadas — desde que não estejam bloqueadas ou em pane permanente.

C. Mecanismo CDC-P (Preempção Cooperativa)

Para emergências persistentes, o CDC-P permite reduzir temporariamente o período de uma tarefa para 1 tick, conferindo-lhe prioridade temporal máxima. A ativação é feita por software (não por interrupção de timer), de forma explícita e rastreável. A tarefa preemptiva ainda executa em seu

ponto definido na sequência do despachador — preservando o determinismo espacial.

D. Latência Medida

Tabela II
LATÊNCIA DE RESPOSTA A EVENTOS NO CDC-P (PIC16F628A, 4 MHz)

Mecanismo	Latência (ticks)	Tempo (8 ms tick)
URG-S (evento pontual)	0	0–112 μ s (overhead do despachador)
CDC-P (emergência)	1	8 ms
Tick dinâmico (2 ms)	1	2 ms

O overhead do despachador completo, medido experimentalmente, é de 112 μ s a 4 MHz — equivalente a 1,4% do tick padrão de 8 ms.

V. COMPARAÇÃO EXPERIMENTAL

A. Plataforma de Teste

- **Microcontrolador:** PIC16F628A (Microchip)
- **Arquitetura:** Harvard 8-bit, 35 instruções
- **Clock:** 4 MHz (1 MIPS)
- **Memória:** 2048 palavras Flash, 224 bytes RAM, 128 bytes EEPROM
- **Stack:** 8 níveis de hardware
- **Compilador:** CCS C (especializado para PIC)

B. Métricas do CDC-P v3.1

Tabela III
RECURSOS UTILIZADOS PELO CDC-P v3.1 (10 TAREFAS)

Recurso	Utilizado	Disponível	Ocupação
ROM	1360 palavras	2048 palavras	66%
RAM (pior caso)	86 bytes	224 bytes	38%
Stack	4 níveis	8 níveis	50%
Overhead/tick	112 μ s	8000 μ s	1,4%

C. Comparação com FreeRTOS

Tabela IV
CDC-P VS FREERTOS — ESTIMATIVAS PARA 10 TAREFAS

Métrica	CDC-P (PIC16)	FreeRTOS (ARM CM)
ROM do kernel	2,1 KB	10–15 KB
RAM (10 tarefas)	80 bytes	2000–3000 bytes
Pilha total	4 níveis (hw)	2000+ bytes
Prioridade	Tridimensional	Unidimensional
Diagnóstico de pane	Nativo	Inexistente
Condições de corrida	Imune	Possíveis
Deadlocks	Imune	Possíveis
Inversão de prioridade	Imune	Possível

D. Análise

O CDC-P utiliza 5–7x menos ROM e 25–37x menos RAM que um RTOS preemptivo típico. A imunidade a condições de corrida, deadlocks e inversão de prioridade não é obtida por mecanismos de correção, mas por construção arquitetural: as tarefas não são interrompidas, portanto não há concorrência real.

VI. IMPLICAÇÕES PARA SISTEMAS CRÍTICOS

A. Certificação

Normas como DO-178C (aviação), IEC 61508 (segurança funcional) e ISO 26262 (automotivo) exigem evidência de determinismo temporal e ausência de comportamentos imprevisíveis. Sistemas com preempção assíncrona requerem análise exaustiva de pior caso (WCET) e verificação de todas as interações entre tarefas — um problema de complexidade combinatorial.

O CDC-P, por sua arquitetura síncrona e cooperativa, reduz significativamente o esforço de certificação:

- A ordem de execução é fixa e imutável;
- O WCET de cada tarefa pode ser medido experimentalmente pelo próprio sistema;
- Não há interações não-previstas entre tarefas;
- O sistema de 7 camadas de proteção provê contenção de falhas verificável.

B. Previsibilidade como Propriedade Arquitetural

A tese central do CDC-P é que a previsibilidade não deve ser um objetivo de verificação, mas uma **propriedade emergente da arquitetura**. Quando o sistema é projetado para ser previsível por construção, a verificação torna-se confirmação, não descoberta.

C. Imunidade a Condições de Corrida por Construção

Tabela V
IMUNIDADE ARQUITETURAL VS MITIGAÇÃO PALIATIVA

Falha	RTOS Preemptivo	CDC-P
Condição de corrida	Mutex, semáforos	Imune (sem interrupção)
Deadlock	Priority ceiling	Imune (sem locks múltiplos)
Inversão de prioridade	Priority inheritance	Imune (sem preempção)
Loop infinito	Watchdog externo	Detectável (watchdog + Xerife)

VII. CONCLUSÃO

A. A Preempção Síncrona é Tecnicamente Superior

Os resultados experimentais demonstram que a preempção síncrona, implementada no CDC-P, supera a preempção assíncrona em todas as métricas relevantes para sistemas embarcados críticos: consumo de memória, previsibilidade temporal, imunidade a falhas de concorrência e facilidade de certificação.

B. A Adoção da Preempção Assíncrona é Motivada por Mercado

A predominância da preempção assíncrona na indústria não encontra justificativa técnica. Trata-se de um modelo de negócio que monetiza a complexidade: ferramentas de debug, consultorias de certificação, treinamentos e licenças de RTOS constituem um ecossistema bilionário que se beneficia da dependência tecnológica dos desenvolvedores.

C. Trabalhos Futuros

- Porte do CDC-P para arquiteturas RISC-V (CH32V003, CH32V307);
- Implementação em Rust com garantias de segurança de memória;
- Síntese em hardware (FPGA/ASIC) da máquina de estados do despachador;
- Estudo de caso de certificação DO-178C com CDC-P.

D. Considerações Finais

Parafraseando Dijkstra [2]: “A simplicidade é um pré-requisito para a confiabilidade.” O CDC-P demonstra que é possível construir sistemas embarcados críticos com uma fração dos recursos normalmente considerados necessários — desde que se projete para a correção, e não para a correção posterior.

A crítica definitiva à indústria de software moderno permanece: “A bagunça tolerada por abundância de recursos.”

REFERÊNCIAS

- [1] E. W. Dijkstra. “Go To Statement Considered Harmful.” *Communications of the ACM*, vol. 11, no. 3, pp. 147–148, 1968.
- [2] E. W. Dijkstra. “The Humble Programmer.” *Communications of the ACM*, vol. 15, no. 10, pp. 859–866, 1972.
- [3] C. A. R. Hoare. “The Emperor’s Old Clothes.” *Communications of the ACM*, vol. 24, no. 2, pp. 75–83, 1981.
- [4] N. Wirth. “A Plea for Lean Software.” *IEEE Computer*, vol. 28, no. 2, pp. 64–68, 1995.
- [5] D. E. Knuth. “Structured Programming with go to Statements.” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [6] E. A. Lee. “Embedded Software.” *Advances in Computers*, vol. 56, pp. 55–95, Academic Press, 2002.
- [7] FreeRTOS Project. “FreeRTOS Market Leading RTOS for Embedded Systems.” <https://www.freertos.org>. Acesso em: julho 2026.
- [8] M. R. Braga. “CDC-P v3.1: Um Cyber Organismo com Previsibilidade Absoluta.” Repositório GitHub, 2026.

APÊNDICE A MÉTRICAS DETALHADAS DO CDC-P

Tabela VI
EVOLUÇÃO DO CONSUMO DE RECURSOS — V2.X A V3.1

Métrica	v2.x	v3.0	v3.1
ROM (words)	936	1188	1360
RAM (bytes)	75	78	80
Stack (níveis)	4	4	4
Overhead (μ s)	80–100	112	112

Tabela VII
TEMPOS DE EXECUÇÃO DE OPERAÇÕES COMUNS (PIC16F628A, 4 MHz)

Operação	Ciclos	Tempo
Ler/Escriver pino digital	1–3	1–3 μ s
Soma/Subtração 8 bits	1–2	1–2 μ s
Multiplicação 8×8	~50	~50 μ s
Divisão 8/8	~100	~100 μ s
Leitura ADC (1 canal)	~20	~20 μ s
Enviar 1 byte serial (9600 bps)	–	~104 μ s
PID (3 termos)	~300	~300 μ s
LCD 16×2 (tela inteira)	~6400	~6,4 ms
LCD 128×64 SPI (tela)	~30720	~30,7 ms
Gravação EEPROM (1 byte)	~10000	~10 ms
CRC-16 (1 byte)	~50	~50 μ s

Tabela VIII
TABELA DE DECISÃO PARA ESCOLHA DE PERÍODO E TICK

WCET da Tarefa	Tick 2ms	Tick 8ms	Tick 50ms	Ação
Até 100 μ s	●	●	●	taskX = 1
100–500 μ s	●	●	●	taskX = 1
500–1800 μ s	●	●	●	taskX ≥ 2
1,8–7,8 ms	●	●	●	taskX ≥ 4
7,8–15 ms	●	●	●	tick 50ms
15–49 ms	●	●	●	tick 50ms
Acima de 49 ms	●	●	●	DIVIDIR!

● Verde: cabe com folga. ● Amarelo: cabe, requer cuidado. ● Vermelho: não cabe.

APÊNDICE B TEMPOS TÍPICOS DE OPERAÇÕES E DIMENSIONAMENTO APÊNDICE C ESTIMATIVAS DE PORTABILIDADE

Tabela IX
CDC-P EM DIFERENTES PLATAFORMAS

Plataforma	Clock	Ganho	Tick Mín.
ATmega328P	16 MHz	4x	1 ms
STM32F103	72 MHz	60–70x	50 μ s
RP2040	133 MHz	120x	50 μ s
CH32V003	48 MHz	48x	100 μ s
ESP32	240 MHz	200–240x	10 μ s